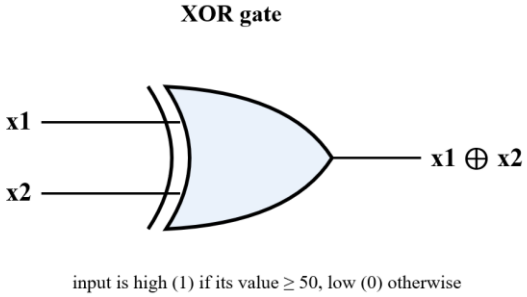
Architetture dei Sistemi di Elaborazione **Laboratory 7**	Delivery date: 18th November 2026 Expected delivery of lab_07.zip must include: - zipped project folder of the exercises 1 and 2 - this document compiled possibly in pdf format.

Exercise 1)

TinyML brings neural networks to microcontrollers. In this laboratory you will run the inference of a very small neural network, TinyXOR, on the ARM core. TinyXOR has 2 inputs, one hidden layer of 2 ReLU neurons and 1 output neuron (a 2-2-1 network with only 9 integer parameters) and it was trained to reproduce the XOR logic gate: its output must be 1 when exactly one of the two inputs is high (≥ 50), 0 otherwise. Inputs are integers in the range 0–100 (fixed-point representation of 0.00–1.00). Write a program in ARM assembly language that runs the network on a test dataset and evaluates its accuracy.



Truth table

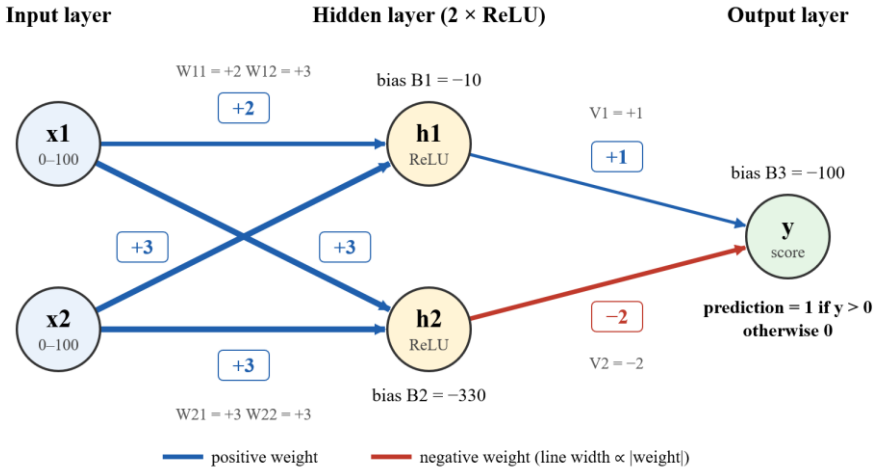
x1	x2	x1 ⊕ x2
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

For each test case, i.e., a pair of inputs (x1, x2), the network computes:

```
h1 = ReLU(W11·x1 + W12·x2 + B1)
h2 = ReLU(W21·x1 + W22·x2 + B2)
y = V1·h1 + V2·h2 + B3
prediction = 1 if y > 0, otherwise 0
ReLU(v) = v if v > 0, otherwise 0
```

The model parameters are:

Parameter	W11	W12	B1	W21	W22	B2	V1	V2	B3
Value	2	3	-10	3	3	-330	1	-2	-100



The parameters must be defined as constants with the EQU directive and loaded into registers only through the LDR Rd, =#CONSTANT instruction.

The test dataset contains 8 test cases. Each test case has a unique ID: it is only a number used to find the data of that test case in memory, and it is not related with the model. The two inputs and the expected output of each test case are stored in three separate tables as <ID, value> pairs. These pairs are shuffled and include some extra pairs that are not part of the test, so to process a test case you must search each table for its ID:

Test IDs	DCD	0x1A, 23, 0x2F, 77, 0x40, 130, 0x99, 201
Label	DCD	47, 1, 0x40, 0, 201, 1, 0x82, 0, 0x17, 1, 0x99, 1, 77, 0, 26, 0, 0x55, 1
Input X1	DCD	130, 60, 0x99, 52, 0x55, 40, 26, 5, 0xC9, 8, 0x17, 97, 77, 95, 0x40, 30, 47, 12
Input X2	DCD	0x4D, 99, 201, 74, 64, 35, 0x1A, 8, 0x82, 70, 47, 91, 153, 1, 23, 3, 0x3C, 50
#Tests	DCB	8

Where:

- Test IDs is the list of the IDs of the test cases to evaluate (one 4-byte word each):
  - ID1, ID2, ID3, …, ID8
- Label is a table of <ID, label> pairs: the ID of a test case (4 bytes) followed by the expected (true) output of the XOR gate for that test case (4 bytes):
  - **0:** FALSE (both inputs low or both inputs high)
  - **1:** TRUE (exactly one input high, i.e., ≥ 50)
  - Layout in memory: <ID, label>, <ID, label>, … (in any order)
- Input X1 is a table of <ID, x1> pairs: the ID of a test case (4 bytes) followed by the value of its first input x1 (0–100).
- Input X2 is a table of <ID, x2> pairs: the ID of a test case (4 bytes) followed by the value of its second input x2 (0–100).
- #Tests is a 1-byte constant that indicates the number of test cases (i.e., the number of entries in Test IDs).
  For example, the first test case has ID 0x1A (= 26). Searching the tables for this ID gives x1 = 5 (Input X1), x2 = 8 (Input X2) and label = 0 (Label). Both inputs are low, so the expected output is 0.

Run the network on every test case and count the test cases that are correctly classified (prediction equal to the label); store the result in register R11. Then, for each of these test cases, compute the confidence of the network: the absolute value |y|, sum it to the other correctly classified test cases, and store it in register R10.

Note: The constant data section must be defined in the code section, with a 2-byte alignment and 4096 boundary zero bytes.
Example:

```
…
// ALIGNMENT
// BOUNDARY (SPACE ….)
MY DATA
// BOUNDARY (SPACE ….)
…
```

Exercise 2)

Store the IDs of the test cases in three separate vectors, CORRECT, FALSE_POS, and FALSE_NEG, based on the outcome of the classification (CORRECT: prediction equal to the label; FALSE_POS: prediction 1 but label 0; FALSE_NEG: prediction 0 but label 1). Within each vector, sort the IDs in ascending order according to the (signed) network output y.

Then, save in R11 and R12 the ID and the output y of the misclassified test case on which the network was most confident (largest |y| among FALSE_POS and FALSE_NEG).

Compute the needed bytes for the above vectors.

Vector	**Size [bytes]**
CORRECT	
FALSE_POS	
FALSE_NEG	

Report the following program characteristics (Hint: See the build output window in Keil).

	Size [bytes]
Program Size	
Read Only data	
Read Write data	
Zero Initialized data	

And provide a brief explanation about which directives can influence the previous program characteristics. Finally, inspect the disassembly and indicate which model parameters were placed in the literal pool and which were encoded directly as immediate operands (e.g. MOV/MVN), explaining why.
